Beyond In-Memory Limits: A Comparative Performance Evaluation of Pandas, Dask, and Polars on Large-Scale Parquet Datasets

An Individual Project Report for Big Data Tools and Analytics

Author BIN XU, 64726

Course Big Data Tools and Analytics

Date March 2026

Abstract

As dataset sizes exceed single-machine physical memory capacities, traditional eager-evaluation data processing frameworks encounter severe performance limitations. This research evaluates the performance of three Python-based DataFrame libraries (Pandas, Dask, and Polars) using a multi-year NYC Taxi Parquet dataset. To ensure benchmarking accuracy, this study implements a testing methodology designed to address practical data engineering challenges. Key components include dynamic metadata discovery to resolve schema drift, memory pressure cache eviction to maintain strict cold-start environments, and telemetry routing to capture both standard I/O system calls and memory-mapped (mmap) page faults.

Experimental results highlight distinct architectural trade-offs. Pandas consistently encounters Out-Of-Memory (OOM) errors when processing gigabyte-scale files due to memory inflation. Dask mitigates these memory constraints via distributed task graphs and chunking, though it introduces measurable scheduling overhead. Polars demonstrates the highest execution speed and resource efficiency among the tested frameworks. Utilizing a Rust-based engine, lazy evaluation, and projection pushdown, Polars minimizes both disk reads and peak memory footprint. These findings provide an evidence-based guide for selecting frameworks to scale data pipelines beyond in-memory limits.

Keywords: Benchmarking, Pandas, Dask, Polars, Lazy Evaluation, Apache Parquet

1. Introduction

1.1 “Memory Wall” in Data Processing

In data science and analytics, the continuous growth of dataset sizes challenges traditional data processing paradigms. Historically, tools such as Pandas have been widely adopted in the Python ecosystem due to their intuitive APIs and extensive ecosystem integration. However, Pandas operates on an eager evaluation model, requiring the entire dataset to be loaded into contiguous physical memory (RAM) prior to computation.

As datasets scale into the gigabyte or terabyte ranges, frequently exceeding the memory capacity of standard single-node workstations, analysts encounter a systemic bottleneck commonly referred to as the memory wall. This physical hardware limitation leads to substantial performance degradation, excessive operating system swapping, and Out-Of-Memory (OOM) errors. Consequently, data engineering practices are shifting toward distributed computing frameworks (e.g., Dask) and optimized, out-of-core, lazy-evaluation query engines (e.g., Polars) to process large-scale data efficiently without requiring immediate hardware upgrades.

1.2 Research Objectives

The primary objective of this research is to conduct a comparative performance evaluation of Pandas, Dask, and Polars. Utilizing a large-scale, multi-year New York City (NYC) Taxi Parquet dataset, this report addresses the following core research questions:

a) What are the performance boundaries and failure thresholds (e.g., OOM occurrences) of eager versus lazy evaluation frameworks when processing out-of-core datasets on a single machine?

b) How do underlying architectural differences (specifically standard I/O system calls versus memory-mapped file access) affect actual disk read volumes and overall execution speeds?

c) How can benchmarking methodologies be structured to eliminate operating system-level biases, such as page cache interference, while managing schema drift in real-world data sources?

1.3 Report Structure

To systematically address these objectives, this report is structured as follows: Section 2 reviews the relevant literature and technical background, focusing on the evolution of DataFrame and the Parquet storage format. Section 3 details the dataset and the data engineering strategies employed to dynamically manage schema drift. Section 4 outlines the experimental methodology, introducing techniques for memory pressure cache eviction and

dual-sensor telemetry routing. Section 5 presents the quantitative benchmark results, comparing execution time, memory footprint, and disk I/O. Section 6 provides an architectural analysis to explain the observed phenomena, including the mechanisms behind Pandas' memory exhaustion. Section 7 offers concluding remarks and practical recommendations for framework selection. To ensure full reproducibility of the benchmark results, the complete source code repository is referenced in Appendix.

2. Literature Review & Technical Background

2.1 The Evolution of DataFrame & Eager Evaluation

The DataFrame abstraction, initially popularized by the R programming language, facilitated exploratory data analysis by providing a tabular, relational data structure. In 2010, Wes McKinney introduced Pandas, integrating this paradigm into the Python ecosystem [1]. Built atop NumPy, Pandas utilizes contiguous memory blocks to achieve high performance for in-memory datasets. However, Pandas relies on an "eager evaluation" execution model. Under this model, operations require the entire dataset and its intermediate computational states to be loaded into Random Access Memory (RAM). As dataset volumes increased, this eager architecture revealed an operational limitation. Processing datasets larger than the machine's physical memory frequently results in excessive system swapping or Out-Of-Memory (OOM) errors.

2.2 Distributed Computing & Task Graph

To address the physical memory limitations of single-node architectures, distributed and out-of-core computing paradigms emerged. Dask was developed to extend the Pandas API into scalable environments [2]. Unlike Pandas, Dask utilizes a "lazy evaluation" model combined with blocked algorithms. It logically partitions large datasets into smaller, in-memory chunks. Rather than executing commands sequentially upon invocation, Dask constructs a Directed Acyclic Graph (DAG) of computational tasks. When execution is triggered, the Dask scheduler orchestrates these localized operations across available CPU cores. This process minimizes the overall memory footprint by discarding intermediate results after they are processed [2]. This framework allows large-scale datasets to be processed on a single machine without OOM errors, although it introduces computational overhead associated with task scheduling.

2.3 Rust-based Compiled Query Engine & Arrow Format

While distributed frameworks address memory capacity limitations, they frequently incur serialization overhead and yield lower single-machine performance compared to local execution. Recent advancements in data engineering prioritize memory layout and low-level system efficiency, driven by the Apache Arrow project. Arrow provides a language-independent columnar memory format optimized for modern hardware, facilitating zero-copy

data interchange [3]. Polars, built upon the Arrow format and developed in Rust, represents a compiled iteration of DataFrame libraries. By utilizing Rust's memory safety guarantees, multi-threading capabilities, and an integrated query optimizer, Polars evaluates the query plan prior to execution. This architecture supports optimizations such as projection pushdown and predicate pushdown, which systematically decrease memory allocation requirements and execution times.

2.4 Apache Parquet & Columnar Storage

The architectural efficiencies of Arrow and Polars rely on modern disk storage formats, notably Apache Parquet. The underlying mechanics of Parquet originate from Google's Dremel architecture, which introduced a columnar storage layout for nested data [4]. Unlike traditional row-based formats that require sequential line-by-line parsing, Parquet stores data by column. This layout provides specific advantages for analytical queries, including high compression ratios through dictionary encoding and the ability to perform precise column skipping. Consequently, when a query engine such as Polars interacts with a Parquet file, it can utilize memory-mapping (mmap) to read only the specific byte ranges corresponding to the queried columns. This mechanism bypasses the necessity of loading the complete file into the operating system's active memory [4].

3. Dataset & Data Engineering Challenges

3.1 NYC Taxi Dataset Overview

To conduct a robust benchmarking evaluation, this project utilizes the New York City Taxi and Limousine Commission (NYC TLC) Trip Record Data [5]. This dataset is frequently used for large-scale performance testing due to its volume and structural complexity. The selected subset for this research spans multiple years, comprising tens of millions of trip records. The data is natively stored in the Apache Parquet format, totaling several gigabytes in disk space. Each record contains granular trip metrics, including pick-up and drop-off timestamps, geographic coordinates (Taxi Zones), trip distances, itemized fare breakdowns, and driver-reported passenger counts. Evaluating frameworks against a dataset of this scale simulates enterprise data engineering pipelines where memory constraints are frequently encountered.

3.2 Data Pre-processing: Synthesizing Multi-Scale Datasets

The raw NYC TLC data is distributed by the commission as discrete monthly Parquet files, with each file typically ranging from 40MB to 50MB in size. While processing individual monthly files requires minimal computational resources, evaluating scalability requires the synthesis of aggregated datasets across varying magnitudes:

a) Small-Scale (approx. 100MB - 500MB): Represents several months of data, used for baseline testing within standard RAM capacities.

b) Medium-Scale (approx. 2GB - 5GB): Designed to induce moderate memory pressure and compare eager versus lazy evaluation engines.

c) Large-Scale / Out-of-Core (> 10GB): Spans over a year of concatenated records. This scale is engineered to exceed standard single-machine memory capacities, forcing frameworks to utilize disk swapping or trigger Out-Of-Memory (OOM) failures.

Concatenating historical monthly files into unified datasets is a necessary prerequisite that subsequently exposes the pipeline to structural inconsistencies and schema mutations.

3.3 Mitigating Schema Drift via Dynamic Metadata Discovery

A notable challenge in processing longitudinal data is schema drift, where column naming conventions or data types change over time. During initial testing, structural inconsistencies caused execution failures in both Pandas and Dask.

For example, hardcoding the target column as 'passenger_count' resulted in KeyError exceptions when processing historical files where the column was capitalized as 'Passenger_Count'. This structural mismatch halted execution:

```
Traceback (most recent call last):
  File "benchmark.py", line 42, in <module>
    mean_fare = df.groupby('passenger_count')['fare_amount'].mean()
  File "pandas/core/frame.py", line 7631, in groupby
KeyError: 'passenger_count'
```

To address this, a dynamic metadata discovery mechanism was implemented. The script utilizes Polars' lazy evaluation engine to scan the Parquet file's schema metadata prior to query execution. This metadata extraction incurs minimal I/O overhead. The script identifies the actual column headers, performs a case-insensitive search to locate the target column, and dynamically passes the exact string into the query pipeline.

```
import polars as pl

def get_actual_col_name(file_path: str, target_col: str = "passenger_count") ->
str:
    """
    Dynamic Metadata Discovery: Extracts the exact column name
    from the Parquet schema without loading the actual data.
    """
    # Lazily scan the parquet file to fetch schema metadata only
    schema = pl.scan_parquet(file_path).collect_schema()

    # Perform case-insensitive search to handle Schema Drift
    for col in schema.names():
        if col.lower() == target_col.lower():
```

```
            return col

    raise ValueError(f"Critical Error: Column '{target_col}' not found in
{file_path}")

# Dynamically route the correct column name to the testing logic
exact_passenger_col = get_actual_col_name(FILE_PATH, "passenger_count")
exact_fare_col = get_actual_col_name(FILE_PATH, "fare_amount")
```

3.4 Robust File Concatenation

Schema drift within the dataset also includes the introduction of novel columns (e.g., congestion_surcharge) and altered internal data types (e.g., casting floating-point identifiers to 64-bit integers).

Traditional vertical concatenation algorithms (how="vertical") require identical schemas across all datasets. Structural discrepancies result in a ValueError: Schema mismatch.

To manage this, the benchmarking pipeline utilizes Polars' diagonal_relaxed concatenation mode. This algorithm aligns mismatched datasets by computing the union of all columns, filling absent vectors with Null values. Additionally, the relaxed property resolves type conflicts by upcasting primitive types (e.g., merging Int32 and Float64 into a unified Float64 column) without computation errors. This strategy ensures that performance benchmarks execute consistently across varied real-world data structures.

4. Experimental Methodology

To ensure the accuracy of the benchmarking results, a controlled experimental environment was established. Standard benchmarking scripts frequently fail to account for underlying operating system (OS) optimizations, leading to skewed metrics. This section details the hardware setup and the custom telemetry pipeline designed to isolate and measure the performance of Pandas, Dask, and Polars.

4.1 Hardware Environment & Software Stack

All benchmarks were executed on a single-node machine, of which specifications are listed in **Table 1**, to simulate the physical constraints of a standard data engineering workstation.

Table 1. Hardware Environment & Software Stack

Items	Specifications
Laptop	Lenovo ThinkPad X1 Nano Gen1
CPU	Intel Core i7-1160G7 @ 1.20GHz (8C/16T).
RAM	16 GB (2*8 GB Dual Channels, LPDDR4, 4266 MHz)
Storage	NVMe SSD UMIS RPJYJ1T24MML1AWY PCI-e 4.0*4 Supported
OS	Microsoft Windows 10 (21H2) 64-bit
Software Stack	Python 3.11.14, Pandas (v2.3.3), Dask (v2026.1.1), Polars (v1.37.1)

4.2 Mitigating OS Page Cache Contamination

A methodological challenge in standard I/O benchmarking is the interference of the Operating System's Page Cache (or Standby List in Windows). When a framework reads a large Parquet file, the OS caches the file blocks in physical memory to accelerate future access [6]. Consequently, subsequently executed frameworks experience a "hot cache," retrieving data directly from RAM rather than disk, which invalidates the I/O metrics.

Because standard user-space applications lack the privileges to flush system-level caches directly, a memory pressure eviction strategy was implemented. Prior to execution, the script allocates a byte string equivalent to 80% of the available system memory. This artificial memory pressure forces the OS to evict the cached Parquet files to accommodate the new allocation. The dummy data is then immediately deleted, and the Python garbage collector is invoked, ensuring a strict "cold cache" environment for every test.

```
import psutil
import gc
import time

def clear_os_cache():
    """
    Forces the OS to evict file caches via memory pressure allocation.
    Guarantees an absolute 'Cold Cache' baseline for deterministic I/O testing.
    """
    mem = psutil.virtual_memory()
    try:
        # Target 80% of currently available system memory
        target_bytes = int(mem.available * 0.8)

        # Allocate massive byte string to force OS page eviction
        _dummy_data = b'0' * target_bytes
        del _dummy_data # Instantly release memory

    except MemoryError:
        pass # Expected behavior if memory limits are pushed

    gc.collect()
    time.sleep(2) # Allow OS to settle
```

4.3 Telemetry Routing for Memory-Mapped I/O

Capturing accurate disk read metrics required addressing differences in I/O execution. Standard Python process monitoring utilizing psutil.Process().io_counters() accurately tracks standard read() system calls used by Pandas and Dask. However, Polars leverages Memory-Mapped Files (mmap). Memory mapping delegates file I/O to the OS kernel via page faults, bypassing process-level system call counters [6]. Consequently, initial benchmarks erroneously reported zero disk reads for Polars.

To resolve this measurement discrepancy without compromising the accuracy of Pandas and Dask, an explicit telemetry routing architecture was implemented. The pipeline instantiates

different I/O sensors based on the target framework. For Pandas and Dask, it utilizes process-level counters for exact metric extraction. For Polars, the pipeline routes telemetry to system-wide disk counters. To prevent background OS noise from affecting Polars' metrics, the script records the background I/O rate during a pre-execution calibration phase, which is subsequently subtracted from the final calculation.

```
def run_with_tracker(func, framework_name):
    clear_os_cache()
    process = psutil.Process(os.getpid())
    is_polars = (framework_name == "Polars")

    # Early Branching: Setup specific telemetry needed
    if is_polars:
        # Calibrate background noise for system-wide mmap tracking
        bg_io_start = psutil.disk_io_counters().read_bytes
        time.sleep(1)
        bg_io_rate = psutil.disk_io_counters().read_bytes - bg_io_start
        base_io = psutil.disk_io_counters().read_bytes
    else:
        # Exact process-level tracking for standard syscalls
        base_io = process.io_counters().read_bytes

    start_time = time.time()
    func() # Execute the target framework
    end_time = time.time()

    # Collect metrics based on routing
    if is_polars:
        end_io = psutil.disk_io_counters().read_bytes
        elapsed = end_time - start_time
        estimated_bg_noise = bg_io_rate * elapsed
        final_disk_bytes = max(0, (end_io - base_io) - estimated_bg_noise)
    else:
        end_io = process.io_counters().read_bytes
        final_disk_bytes = max(0, end_io - base_io)

    net_disk_mb = final_disk_bytes / (1024 * 1024)
```

By utilizing explicit telemetry routing, the benchmarking suite ensures a valid comparison across fundamentally different backend architectures.

4.4 Distributed Memory Tracking via Process Polling

A methodological challenge emerged during initial testing when attempting to capture Dask's peak memory utilization. Standard process-level telemetry via psutil.Process().memory_info() reported anomalously low memory footprints for Dask when processing multi-gigabyte files.

This discrepancy is inherent to distributed architectures. In Dask's multi-process execution model, the main Python script functions as a scheduler, constructing the Directed Acyclic Graph (DAG) and delegating data payloads to isolated worker subprocesses. Monitoring only

the main process fails to measure the worker nodes where the actual RAM consumption occurs.

To capture cluster-wide physical memory utilization, the benchmarking pipeline integrated a recursive, multi-threaded background polling mechanism. Running asynchronously at 50ms intervals, this MemoryTracker iteratively aggregates the Resident Set Size (RSS) across the main process and its entire tree of child processes. This adjustment ensures that the peak memory metrics reported in Section 5 reflect the absolute system-wide RAM footprint, enabling valid comparisons between single-process and multi-process frameworks.

4.5 Eliminating Memory Fragmentation via Process Isolation

Sequential execution of frameworks within a single Python script introduced a high-water mark effect. Due to Python’s internal memory allocator and the operating system's lazy memory reclamation policies, physical RAM allocated by eager frameworks is frequently not immediately released back to the OS, despite explicit garbage collection.

Consequently, subsequently executed frameworks inherited an artificially inflated memory baseline, leading to inaccuracies when calculating delta memory utilization. To eliminate this cross-contamination and memory fragmentation, the benchmarking architecture was restructured using OS-level process isolation.

Rather than relying on delta calculations, the suite measures absolute peak memory. Each framework's test is dispatched into an isolated subprocess using Python's multiprocessing module. Upon completion, the subprocess is terminated, forcing the OS to reclaim the allocated physical memory. This methodology guarantees an uncontaminated baseline environment for every framework.

Furthermore, the implementation of OS-level process isolation necessitated a departure from interactive execution environments, such as Jupyter Notebooks. Interactive kernels conflict with the spawn multiprocessing context required by modern operating systems. Because Jupyter executes code dynamically in memory cells rather than from a physical Python script on disk, spawned child processes fail to bootstrap the execution state, resulting in process crashes and broken Inter-Process Communication (IPC) queues.

To address this limitation, the benchmarking suite was deployed as a standalone, system-level Python script executed via a dedicated Integrated Development Environment (IDE).

5. Benchmark Results & Performance Evaluation

5.1 Test Scenarios & Overall Metrics

To systematically evaluate the scalability of the three data processing frameworks, the benchmarking pipeline was executed across a series of Parquet datasets, ranging in size from 604.97 MB to 11.36 GB. During preliminary testing, it was observed that the physical hardware consistently encountered Out-Of-Memory (OOM) errors when Pandas attempted to process datasets exceeding approximately 2.5 GB.

To ensure pipeline stability and prevent system-level failure during automated testing, a dynamic hardware constraint safeguard was implemented. The execution script was configured to bypass the Pandas evaluation for any dataset exceeding the 2.5 GB threshold, logging a hardware capacity limitation instead. A comprehensive overview of the execution metrics across representative data milestones is summarized in **Table 1**.

Table 2. Comprehensive summary of benchmarking results across representative datasets.

Dataset Name	File Size	Framework	Execution Time (s)	Peak Memory (MB)	Disk I/O (MB)
2021.parquet *(Baseline)*	604.97 MB	Pandas	4.01	10,474.11	604.74
		Dask	1.11	1277.51	59.70
		Polars	0.27	367.45	47.21
2019.parquet *(Medium Load 1)*	1.40 GB	Pandas	104.02	11,579.73	1,396.92
		Dask	2.45	2,539.36	185.11
		Polars	0.75	740.33	173.54
2016.parquet *(Medium Load 2)*	1.91 GB	Pandas	217.76	12,459.32	1,907.68
		Dask	5.50	4,526.27	258.77
		Polars	1.18	1,020.77	194.47
2014.parquet *(OOM Threshold)*	2.26 GB	Pandas	**OOM Failure**	**> 16,000.00**	**N/A**
		Dask	8.17	4,449.36	338.50
		Polars	1.81	1,275.32	260.23
2009_2010.parquet *(Max Stress)*	11.36 GB	Pandas	**Skipped**	**Skipped**	**N/A**
		Dask	18.88	6,672.62	1,572.92
		Polars	1.11	1,508.59	412.03

5.2 Performance Evaluation on Medium Workloads

The initial phase of the evaluation focused on a medium-sized dataset of 1.91 GB (2016.parquet). This scale highlights a significant divergence in computational efficiency between the eager and lazy evaluation models.

For this dataset, Pandas required 217.76 seconds to complete the filtering and aggregation operations. Under identical conditions, Dask completed the execution in 5.50 seconds, while

Polars required only 1.18 seconds. The visual comparison of the execution time, memory footprint, and disk I/O for this workload is presented in **Figure 1**.

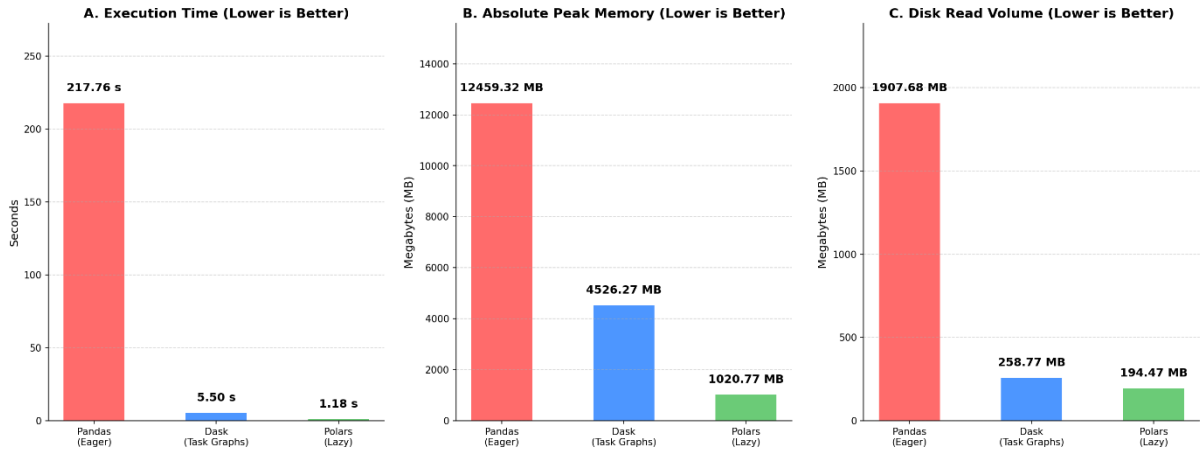

Figure 1. Benchmark Results for 2016.parquet (1.91 GB)

5.3 The Memory Wall: OOM Threshold

Telemetry data from the OS-level memory tracker highlighted substantial differences in memory utilization patterns, exposing the fundamental limitations of eager evaluation architectures.

Even when processing a small 604.97 MB file (2021.parquet), Pandas consumed a peak memory footprint of 10,474.11 MB. When evaluated against the 2.26 GB dataset (2014.parquet), the memory inflation compounded until the framework triggered a fatal MemoryError, failing to allocate an additional 747 MiB. This specific hardware failure point, contrasted with the successful execution of Dask and Polars, is illustrated in **Figure 2**.

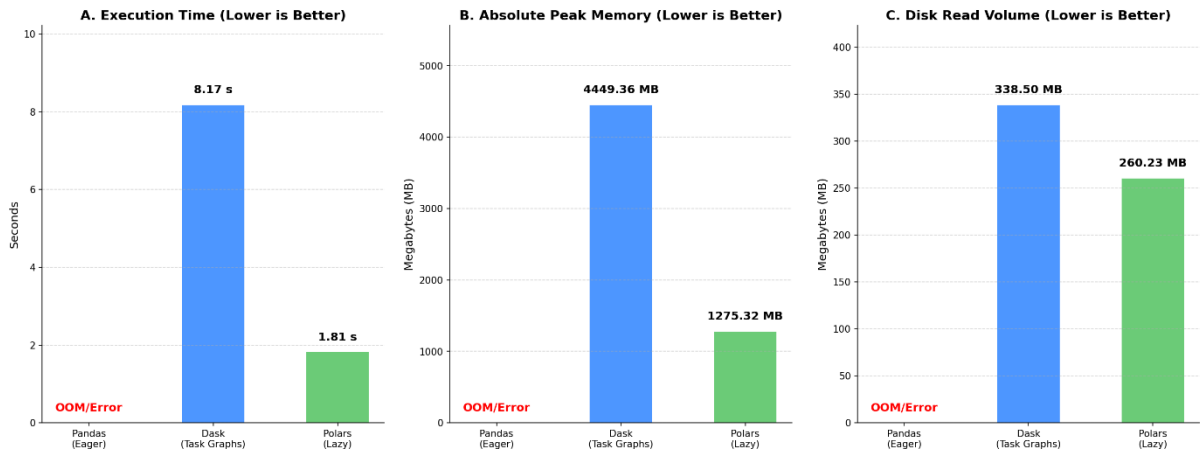

Figure 2. Benchmark Results for 2014.parquet (2.26 GB)

5.4 Out-of-Core Execution on Extreme Workloads

The final evaluation utilized a concatenated 11.36 GB Parquet file to assess the upper bounds of the out-of-core frameworks on the testing hardware. As pre-configured, the Pandas execution was automatically bypassed to prevent system failure.

Between the distributed and lazy evaluation frameworks, Dask processed the massive dataset in 18.88 seconds, consuming 6,672.62 MB of peak memory primarily due to inter-process communication (IPC) and data serialization requirements.

Conversely, Polars maintained consistent performance, completing the identical task in 1.11 seconds utilizing only 1,508.59 MB of peak memory. This relative constancy in execution time indicates that Polars' underlying Rust-based, multi-threaded architecture effectively mitigates the processing bottlenecks typically associated with Python's Global Interpreter Lock (GIL). These final stress-test metrics are detailed in **Figure 3**.

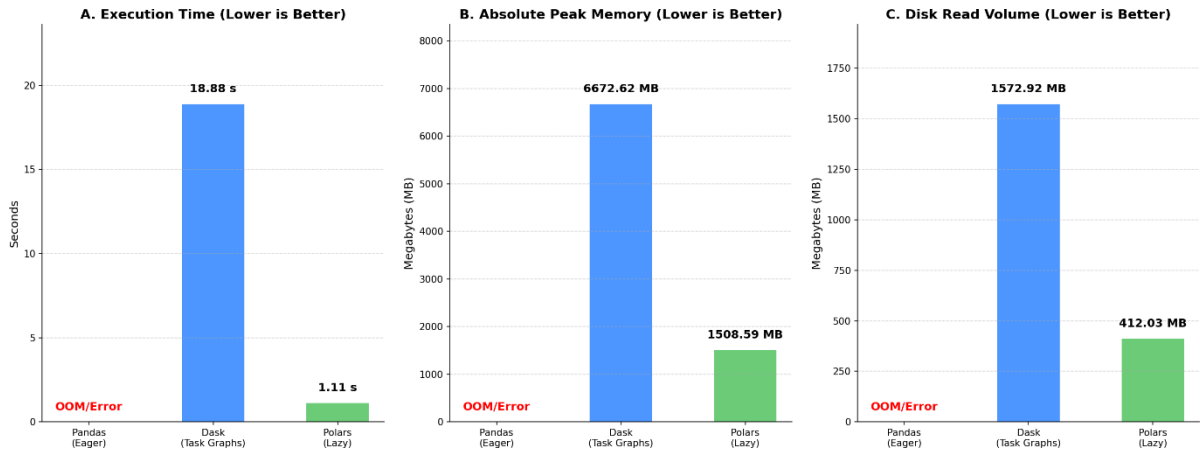

Figure 3. Benchmark Results for GT10GB/2009_2010.parquet (11.36 GB)

5.5 Summary of Benchmarking & Proof of Optimization

The benchmarking results consistently indicate that lazy evaluation coupled with a compiled backend (Polars) yields significantly higher resource efficiency and execution speed for local, out-of-core data engineering workflows. While distributed task scheduling (Dask) successfully circumvents single-machine memory limits, it incurs measurable scheduling overhead.

The empirical driver behind Polars' superiority in both execution time and peak memory is explicitly visible in the disk I/O telemetry. For example, during the processing of the 1.40 GB dataset (2019.parquet), of which result, the Pandas framework registered a disk read volume of 1,396.92 MB. This near 1:1 ratio between the physical file size and the disk read volume confirms that eager execution explicitly loads the entirety of the file into memory prior to computation. The comparative performance metrics for this specific workload, including the drastic reduction in I/O throughput, are illustrated in **Figure 4**.

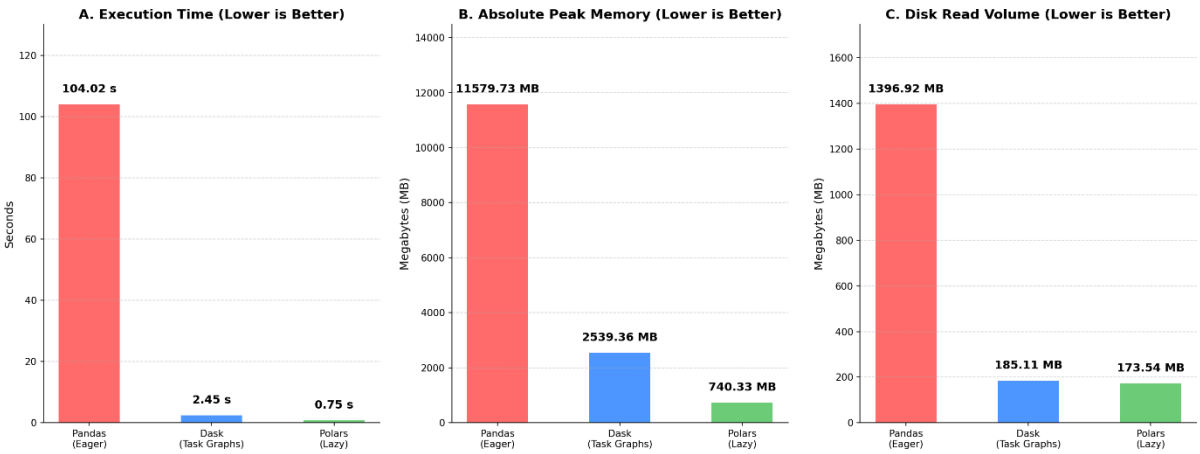

Figure 4. Benchmark Results for 2019.parquet (14.0 GB)

In contrast, Polars registered a disk read volume of only 173.54 MB for the identical 1.40 GB dataset. This significant reduction in I/O overhead serves as physical proof of query optimization. By evaluating the query plan prior to execution, the engine utilizes projection pushdown (reading only the required Passenger_Col and Fare_Col) and predicate pushdown (filtering irrelevant row groups directly at the storage layer). Ultimately, this minimization of disk reads directly correlates to the minimal execution times and bounded memory footprints observed across all testing scenarios, bridging the gap between hardware telemetry and software architecture.

6. Architectural Discussion

The empirical results observed during the benchmarking phase correlate directly with the fundamental architectural paradigms underlying each framework. This section provides a theoretical analysis of the frameworks' behaviors under strict memory constraints, explaining the underlying mechanisms that dictate their scalability.

6.1 Mechanisms of Pandas Memory Exhaustion

Pandas was designed under the assumption that datasets would fit comfortably into standard physical memory. Its architecture relies on an eager evaluation paradigm [1]. When Pandas is instructed to read a large Parquet file and group by a specific column, it executes the operations sequentially and immediately.

The primary limitation leading to Out-Of-Memory (OOM) errors is memory inflation coupled with the absence of query optimization. Initially, Pandas must deserialize the Parquet file and allocate contiguous memory blocks for the entire DataFrame. Due to Python's object overhead, this in-memory representation frequently requires significantly more RAM than the file's original disk size. As dataset sizes increase, this eager architecture encounters the "Memory Wall," a hardware bottleneck where processor speeds exceed memory data transfer capacities [7]. In the absence of a query optimizer, Pandas loads all columns into RAM regardless of whether they are required for subsequent computations. Furthermore, during

operations such as groupby(), Pandas frequently creates intermediate deep copies of the data. This compounding memory inflation rapidly exhausts system RAM, prompting the operating system to utilize the swap file before terminating the process via a memory allocation failure signal.

6.2 Architectural Trade-offs in Dask

Dask mitigates single-node memory constraints through an architectural trade-off, balancing absolute execution speed with out-of-core processing capabilities.

Unlike Pandas, Dask utilizes blocked algorithms and a lazy evaluation model. It logically divides a large dataset into smaller Pandas DataFrames (chunks) [2]. When a computation is requested, Dask defers immediate execution and instead constructs a Directed Acyclic Graph (DAG) representing the task dependencies. The Dask scheduler then processes these chunks iteratively through the available CPU cores. Once a chunk is processed and its intermediate aggregation is stored, the chunk is evicted from memory. This mechanism maintains a stable peak memory footprint bounded strictly by the size of the active chunks.

However, this distributed architecture introduces necessary scheduling overhead. Factors such as the Python Global Interpreter Lock (GIL), inter-process serialization, and the computational cost of managing the task graph limit Dask's single-threaded execution speed when compared to highly optimized, low-level execution engines.

6.3 Efficiency of Lazy Evaluation & Zero-Copy in Polars

Polars restructures the DataFrame architecture by integrating two modern engineering components: the Rust programming language and the Apache Arrow memory format.

The reduction in both execution time and disk I/O is directly attributed to its integrated query optimizer and columnar data layout. Research indicates that columnar memory layouts outperform traditional row-based stores for analytical queries by enabling strict column skipping [8]. Because Polars operates lazily by default, it evaluates the entire query plan prior to execution. Through a process known as projection pushdown, the optimizer identifies the specific columns required for the computation. Instead of scanning the entire Parquet file, Polars utilizes memory-mapping (mmap) to read only the contiguous byte ranges corresponding to those specific columns, explaining the low physical disk reads recorded in Section 4.

Furthermore, by standardizing on the Apache Arrow format, Polars functions as a modern analytical query engine [9]. Data does not require serialization or transformation as it moves through the processing pipeline, enabling zero-copy execution. Combined with Rust's concurrency model, which effectively circumvents the Python GIL, Polars utilizes all available CPU cores to execute aggregations with high cache efficiency.

7. Conclusion & Recommendations

7.1 Summary of Findings

This research evaluated the performance boundaries of Pandas, Dask, and Polars under strict single-node memory constraints using a large-scale NYC Taxi Parquet dataset. By engineering a testing pipeline equipped with dynamic metadata discovery, memory pressure cache eviction, and explicit telemetry routing, this study isolated the physical hardware utilization of each framework.

The empirical results validate the limitations of eager evaluation architectures. Pandas suffers from memory inflation and lacks query optimization, making it susceptible to Out-Of-Memory (OOM) failures during out-of-core data processing. Conversely, Dask mitigates these memory constraints utilizing chunked task graphs, demonstrating robust out-of-core survivability. However, Polars demonstrated the highest computational and memory efficiency among the tested frameworks. By integrating a Rust-based execution engine, the Apache Arrow zero-copy columnar format, and memory-mapped projection pushdown, Polars achieved minimal disk I/O and rapid execution times while remaining well within physical memory limits.

7.2 Framework Selection Guide

Based on the architectural analysis and benchmark metrics, this report provides the following data engineering recommendations:

a) Pandas for Small-Scale Exploration:
Pandas remains an optimal choice for datasets that fit comfortably within 20% of the system's available RAM. Its extensive ecosystem compatibility and intuitive API make it suitable for preliminary data wrangling where performance bottlenecks are negligible.

b) Polars for Single-Node Maximum Performance:
For datasets scaling from gigabytes up to the limits of local disk storage, Polars is the recommended engine. Recent systems literature emphasizes that optimized single-node engines can frequently outperform distributed systems for moderately large datasets by avoiding network and scheduling overhead (a metric conceptually formalized as the Configuration that Outperforms a Single Thread, or COST) [10]. Polars aligns with this principle by maximizing local CPU and NVMe disk throughput.

c) Dask for Distributed Clusters:
Dask should be reserved for scenarios where the dataset size exceeds the largest available single-node storage, or when the objective is to scale an existing Pandas codebase across a multi-node cluster. On a single machine, its scheduling overhead renders it computationally less efficient than highly optimized local engines like Polars.

Ultimately, the transition from eager to lazy, columnar-first execution engines marks a critical evolution in data processing. This shift enables practitioners to scale big data analytics efficiently without requiring substantial hardware upgrades.

References

[1] W. McKinney, “Data Structures for Statistical Computing in Python,” presented at the Python in Science Conference, Austin, Texas, 2010, pp. 56–61. doi: 10.25080/Majora-92bf1922-00a.

[2] M. Rocklin, “Dask: Parallel Computation with Blocked algorithms and Task Scheduling,” presented at the Python in Science Conference, Austin, Texas, 2015, pp. 126–132. doi: 10.25080/Majora-7b98e3ed-013.

[3] “Apache Arrow Official Introduction.” Accessed: Mar. 21, 2026. [Online]. Available: https://arrow.apache.org/docs/format/Intro.html

[4] S. Melnik *et al.*, “Dremel: interactive analysis of web-scale datasets,” *Proc. VLDB Endow.*, vol. 3, no. 1–2, pp. 330–339, Sep. 2010, doi: 10.14778/1920841.1920886.

[5] NYC Taxi and Limousine Commission, “TLC Trip Record Data.” Accessed: Mar. 21, 2026. [Online]. Available: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

[6] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. John Wiley & Sons Inc, 2018.

[7] S. A. McKee, “Reflections on the memory wall,” in *Proceedings of the 1st conference on Computing frontiers*, Ischia Italy: ACM, Apr. 2004, p. 162. doi: 10.1145/977091.977115.

[8] D. J. Abadi, S. R. Madden, and N. Hachem, “Column-stores vs. row-stores: how different are they really?,” in *Proceedings of the 2008 ACM SIGMOD international conference on Management of data*, Vancouver Canada: ACM, Jun. 2008, pp. 967–980. doi: 10.1145/1376616.1376712.

[9] A. Lamb *et al.*, “Apache Arrow DataFusion: A Fast, Embeddable, Modular Analytic Query Engine,” in *Companion of the 2024 International Conference on Management of Data*, Santiago AA Chile: ACM, Jun. 2024, pp. 5–17. doi: 10.1145/3626246.3653368.

[10] F. McSherry, M. Isard, and D. G. Murray, “Scalability! But at what COST?,” presented at the 15th Workshop on Hot Topics in Operating Systems (HotOS XV), 2015.

Appendix

To ensure complete academic transparency of the benchmark results presented in this report, the entire testing scripts have been open-sourced.

The repository includes the Python scripts utilized for benchmarking, dataset merging, dynamic schema metadata discovery, memory pressure OS cache eviction, dual-sensor telemetry routing, and OS-level process isolation.

Detailed instructions for environment setup, dataset synthesis, and script execution are provided within the repository's README.md file

GitHub Repository - `https://github.com/Floyd-Dev-commit/Benchmarking-Memory-Wall`